

 CEFET-MG	SERVIÇO PÚBLICO FEDERAL MINISTÉRIO DA EDUCAÇÃO CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS Departamento de Computação				Nota 18,2
Curso:	<i>Engenharia de Computação</i>	Disciplina:	<i>Modelagem e Desenvolvimento de Software</i>	Turma:	<i>5º Período</i>
Professor:	<i>Edson Marchetti da Silva</i>	2ª Prova		Valor:	<i>25 Pontos</i>
Aluno(a):	<i>Gustavo de Assis Xavier.</i>			Nro:	

1ª Questão – (5 pontos)

Avalie código fonte em Java abaixo, reescreva-o aplicando o princípio da Inversão de Dependência (Dependency Inversion Principle - DIP) do SOLID. O objetivo é melhorar o design do código, desacoplando as classes e tornando o sistema mais flexível e fácil de manter. Desse modo, permitir adicionar novos dispositivos (por exemplo, um aquecedor) sem modificar a classe Switch implementada.

<pre>class Light { public void turnOn() { System.out.println("Luz ligada"); } public void turnOff() { System.out.println("Luz desligada"); } }</pre>	<pre>class Fan { public void turnOn() { System.out.println("Ventilador ligado"); } public void turnOff() { System.out.println("Ventilador desligado"); } }</pre>
<pre>class Switch { private Light luz; private Fan ventilador; public Switch() { this.luz = new Light(); this.ventilador = new Fan(); } public void testar(String dispositivo) { if (dispositivo.equals("LUZ")) { luz.turnOn(); luz.turnOff(); } else { ventilador.turnOn(); ventilador.turnOff(); } } }</pre>	<pre>public class ProvaQuestao { public static void main(String[] args) { Switch s = new Switch(); s.testar("LUZ"); s.testar("Ventilador"); } }</pre> 5,0

2ª Questão (2 pontos)

Informa com a letra associado a um dos padrões GoF com cada uma das definições apresentadas a seguir:

- A. Singleton ✓
B. Iterator ✓
C. State ✓

- D. Observer ✓
E. Command
F. Abstract Factory

- G. Facade ✓
H. Template Method ✓

[G]	Este padrão fornece uma interface unificada para um conjunto de interfaces em um subsistema, facilitando a sua utilização.
[E]	Permite a criação de uma família de objetos relacionados sem especificar suas classes concretas.
[A]	Garante que uma classe tenha apenas uma instância e fornece um ponto global de acesso a essa instância.
[C]	Este padrão permite que objetos mudem seu comportamento quando seu estado interno muda.
[D]	Define uma dependência de um-para-muitos entre objetos, de forma que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.
[F]	Permite que uma operação seja adiada para ser implementada por uma das subclasses.
[H]	Este padrão encapsula uma solicitação como um objeto, permitindo que você parametrize clientes com diferentes solicitações.
[B]	Fornece uma maneira de acessar elementos de uma coleção sequencialmente, sem expor sua representação subjacente.

3ª Questão (5 pontos)



Você foi chamado para construir um web site para vender açaí on-line. Atualmente, a loja física trabalha com o cardápio mostrado ao lado. Ou seja, o cliente escolhe um tamanho base do açaí e pode adicionar quantos toppings desejar. O preço final a ser pago deverá ser calculado a partir do preço base acrescentando o preço dos toppings adicionados. Considerando os padrões GoF, faça:

- a) Qual o nome do padrão que possibilita melhor lidar com o cálculo do valor da compra, considerando que o cliente pode realizar qualquer combinação com 1 dos 3 tamanhos base com mais 12 diferentes toppings? Justifique.

b) Escreva o código para implementar o padrão escolhido.

4ª Questão (1 ponto)

Qual é o impacto da adoção da arquitetura de microsserviços na relação entre arquitetura backend e frontend, juntamente com a arquitetura de API, dentro da Engenharia de Software moderna?

- a) A arquitetura de microsserviços centraliza todas as funcionalidades tanto do backend quanto do frontend, reduzindo a necessidade de arquitetura de API.
- b) Sob a influência da arquitetura de microsserviços, a arquitetura de API se torna obsoleta, pois não há mais necessidade de comunicação entre os diferentes componentes do sistema. ✗
- c) Com a arquitetura de microsserviços, o frontend torna-se responsável por toda a lógica de negócios, enquanto o backend foca apenas no armazenamento de dados, eliminando a necessidade de uma arquitetura de API.
- d) A arquitetura de microsserviços incentiva a separação clara entre backend e frontend, com o backend responsável pela lógica de negócios e processamento de dados, enquanto o frontend cuida da interface do usuário. A arquitetura de API é essencial para a comunicação entre essas camadas.
- e) A arquitetura de microsserviços tem como desvantagem uma forte dependência entre os microsserviços fazendo com que a interrupção de apenas um faz com que toda a aplicação deixe de funcionar.

5ª Questão - (1 ponto)

A Clean Architecture (ou arquitetura limpa), criada por Robert C. Martin, considera como objetivo:

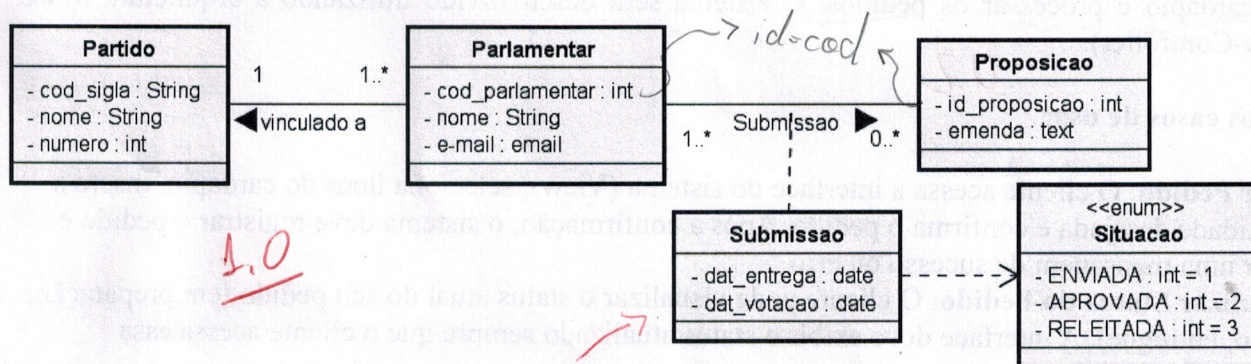
- a) Na arquitetura de software, padronizar e organizar o código desenvolvido, favorecer a sua reusabilidade, assim como a dependência de tecnologia. ✗
- b) Fornecer aos desenvolvedores uma maneira de organizar o código de forma que generalize a lógica de negócios, mantendo-o separado do mecanismo de entrega e dependente do agente externo.

- ✗ Proporcionar independência de interface do usuário e independência de qualquer agente externo, com exceção da dependência de banco de dados específico.
- d) Na arquitetura de software, minimizar os recursos humanos necessários para construir e manter o sistema necessário.
- ✗ e) Configurar as dependências de cada camada sempre em direção à periferia, tendo o core totalmente protegido de qualquer interferência externa ou interna. ✓

6ª) Questão - (2 pontos)

Em uma Casa Legislativa, considere um cenário restrito, no qual uma proposição pode ser submetida por um ou mais parlamentares e um parlamentar pode submeter de nenhuma a várias proposições. As proposições (propostas legislativas) submetidas são avaliadas nas instâncias do Parlamento. Sabe-se ainda que

O modelo de conceitual de classes a seguir modela tal situação:



Em um contexto no qual o modelo conceitual será mapeado segundo a abordagem Mapeamento Objeto-Relacional (ORM), e que as classes “Parlamentar” e “Proposicao” foram mapeadas para um banco de dados relacional da seguinte forma:

Parlamentar ({cod_parlamentar} <PK>, nome, e-mail, cod_sigla <FK>);
 Proposicao ({id_proposicao} <PK>, emenda)

A fim de manter, no modelo lógico de banco de dados relacional, a semântica expressa na classe “Submissao” do modelo conceitual, marque a(s) alternativa(s) que podem ser adotadas para a implementação no banco de dados.

- a) Adicionar na tabela “Parlamentar” a chave estrangeira “id_proposicao”, referenciando a chave primária da tabela “Proposicao”.
- b) Alterar a cardinalidade máxima da associação entre “Parlamentar” e “Proposicao”, com vistas a torná-la com conectividade um-para-muitos.
- ✗ c) Criar a tabela “Submissao” com o seguinte esquema: Submissao ({id_submissao}<PK> , dat_entrega, dat_votacao, cod_parlamentar<FK>, id_proposicao<FK>).
- ✗ d) Criar a tabela “Submissao” com o seguinte esquema: Submissao ({id_proposicao, id_parlamentar}<PK+FK> , dat_entrega, dat_votacao).
- e) Criar a tabela “Submissao” com o seguinte esquema: Submissao (id_submissao , dat_entrega, dat_votacao).

7ª) Questão - (3 pontos)

Um aluno do CEFET após aprovado no processo seletivo é importado para a base de dados do Sistema Acadêmico com a situação de selecionado. Após ele confirmar sua matrícula, o aluno passa a ficar como ativo. Caso ele não se matricule até a data limite da convocação da chamada, ele deve ser automaticamente colocado como desistente, liberando a vaga para a próxima chamada. Um aluno que não conseguir obter sua

diplomação até o período limite do curso será automaticamente colocado como jubilado. O aluno também tem a opção de, a qualquer momento, solicitar o trancamento, o que lhe permite retornar para ativo desde que dentro do seu prazo limite de conclusão. Sabe-se ainda que, um aluno poderá ser expulso da escola, caso cometa algum ato de indisciplina e após ser avaliado por uma comissão e ter o seu direito de defesa garantido, e assim for deliberado. A qualquer momento o aluno pode solicitar sua transferência, o que faz com que perca o direito da vaga. O aluno que conclui todas as disciplinas do curso fica como concluído. A qualquer momento, quando ele consegue o estágio, ele fica estagiando. Finalmente, após finalizar o estágio e ter concluído todas as disciplinas ele pode colar grau e finaliza sua situação como formado.

Elabore um diagrama de estados para mapear a situação de um aluno ao longo de seu ciclo de vida no sistema acadêmico do CEFET.

8ª) Questão - (5 pontos)

Uma empresa está desenvolvendo um sistema de gestão de pedidos para um restaurante. O sistema deve permitir que os usuários façam pedidos, visualizem o status dos pedidos e que os administradores possam gerenciar o cardápio e processar os pedidos. O sistema será desenvolvido utilizando a arquitetura MVC (Model-View-Controller).

4,2

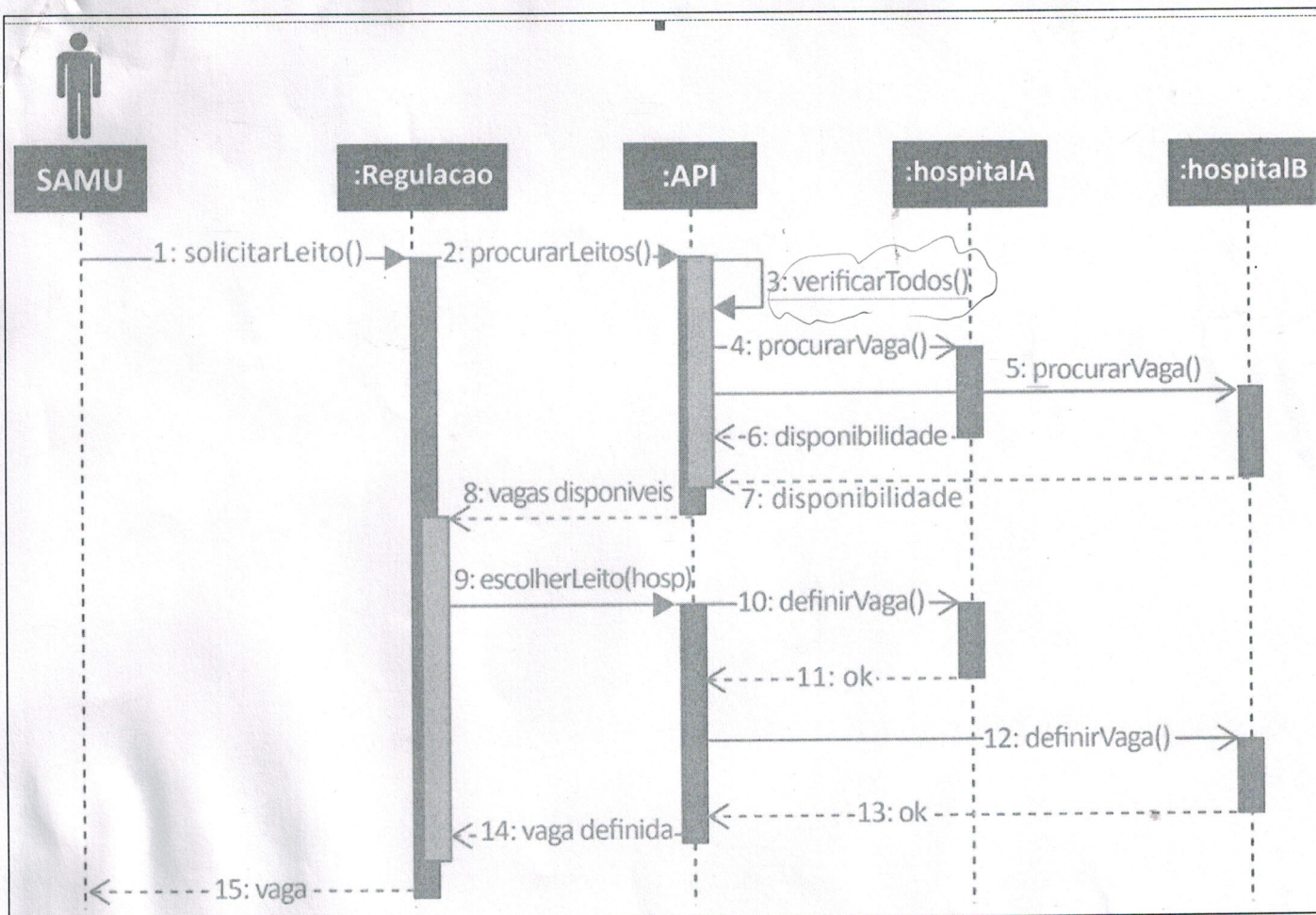
Descrição dos casos de uso:

1. **Fazer Pedido:** O cliente acessa a interface do sistema (View), seleciona itens do cardápio, insere a quantidade desejada e confirma o pedido. Após a confirmação, o sistema deve registrar o pedido e exibir uma mensagem de sucesso ou erro.
2. **Visualizar Status do Pedido:** O cliente pode visualizar o status atual do seu pedido (em preparação, pronto, entregue). A interface deve exibir o status atualizado sempre que o cliente acessa essa funcionalidade.
3. **Gerenciar Cardápio:** O administrador acessa a interface do sistema, onde pode adicionar, remover ou atualizar itens no cardápio. As mudanças devem ser refletidas imediatamente para todos os clientes ao acessarem o cardápio.
4. **Processar Pedido:** O administrador pode visualizar uma lista de pedidos em aberto, selecionar um pedido, marcá-lo como "em preparação", "pronto" ou "entregue". A interface deve permitir a atualização e refletir o status atual do pedido.

Considerando os casos de uso descritos acima, elabore um diagrama de classe modelo de solução com as classes envolvidas em todas as camadas da aplicação informando os métodos mínimos necessários para atender aos casos de uso propostos.

9ª) Questão - Enade (2021) Sistema de Informação – Adaptado (1 ponto)

No desenvolvimento do módulo de integração do sistema do SAMU com os sistemas de hospitais, um analista gerou o seguinte diagrama de sequência.



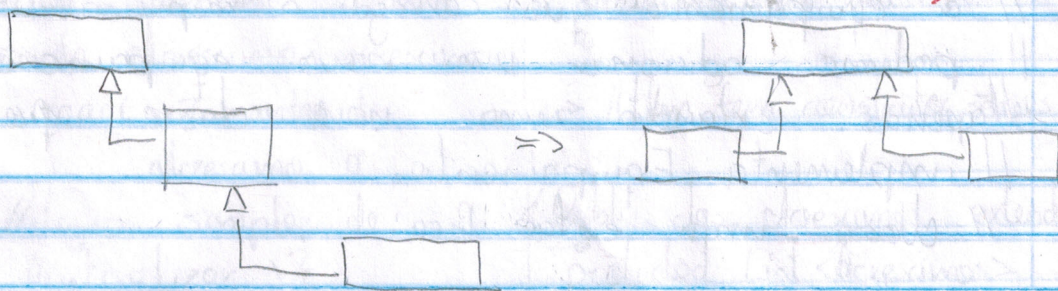
Com relação ao diagrama de sequência apresentado anteriormente, avalie as asserções e assinale a opção FALSA.

- a) A chamada 3:verificarTodos é um método implementado na API. ✓
- b) As chamadas 4: procurarVaga() e 5: procurarVaga() são feitas simultaneamente (em paralelo) pela API para minimizar o tempo de espera da chamada 2 procurarLeitos(). ✓
- ☒ c) Existe um erro no diagrama na linha de vida do método 1:solicitarLeito, pois não há uma representação de uma chamada do método local da classe Regulacao, logo após o retorno (8) da API.
- d) Os métodos 10:definirVaga() e 12:definirVaga() são chamados de forma sequencial.
- e) O método 9:escolherLeito(hosp) é o método da API responsável em dar o feedback aos hospitais se a vaga foi ou não escolhida.

Nome: Gustavo de Assis Xavier

1) O princípio se baseia basicamente em:

5,0



add sem modificar a classe switch!

// podemos fazer uma interface que switch "controla"

```
public interface Equipamento {
    public void turnOn();
    public void turnOff();
}
```

```
class Fan implements Equipamento {
    // continua a mesma, pois já implementa os métodos
}
```

```
// Light também continua a mesma
```

```
class Switch {
    private Equipamento eq;
    public Switch (Equipamento eq) {
        this.eq = eq;
    }
    public void testar () {
        eq.turnOn();
        eq.turnOff();
    }
}
```

// Com isso, invertemos a dependência, e agora
 // o equipamento que carrega a responsabilidade.
 // podemos adicionar um novo equipamento
 // apenas criando uma nova classe que
 // implementa Equipamento.
 // classe main então fica:

```
public class ... {
    public static ... {
        Equipamento eq = new Light();
        Switch s = new Switch(eq);
        s.testar();
        s = new Switch(new Ventilador());
        s.testar();
    }
}
```

3.a) Vamos considerar:

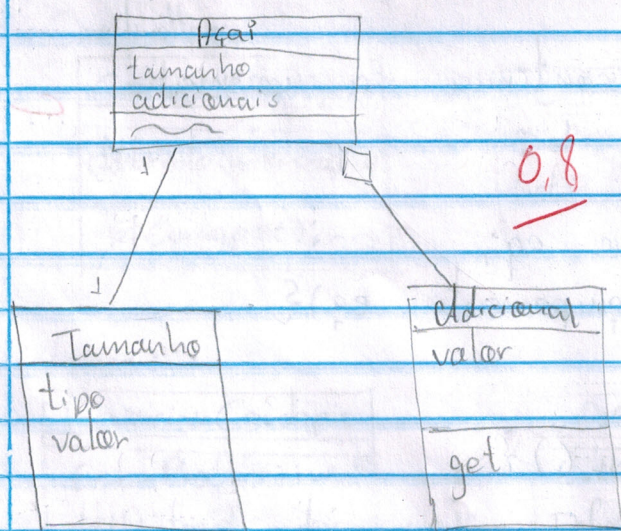
// Temos um açaí, que tem um tamanho e adicionais

// cada adicional

tem seu preço

// cada tamanho

tem seu preço



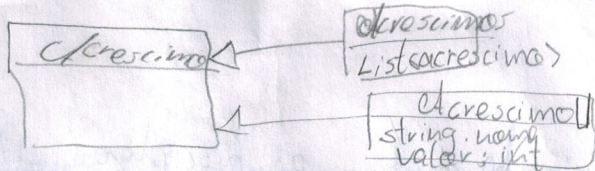
// Dessa forma, podemos

// dizer que um adicional

// pode ser um conjunto

// de um ou mais

// adicionais



2.0

```

Public class acai {
  private Tamanho tamanho;
  private Acrescimo acrescimo;
  public acai(Tamanho t,
               Acrescimo a) {
    // logica simples de
    // inicializar }
}
  
```

```

public class Tamanho {
  // construtor
  private int valor;
  private String tamanho;
}
  
```

7) Diagrama de estados:

```

Public abstract class Acrescimo {
  protected int valor;
}
// tem tem construtor, setters e getters.
  
```

```

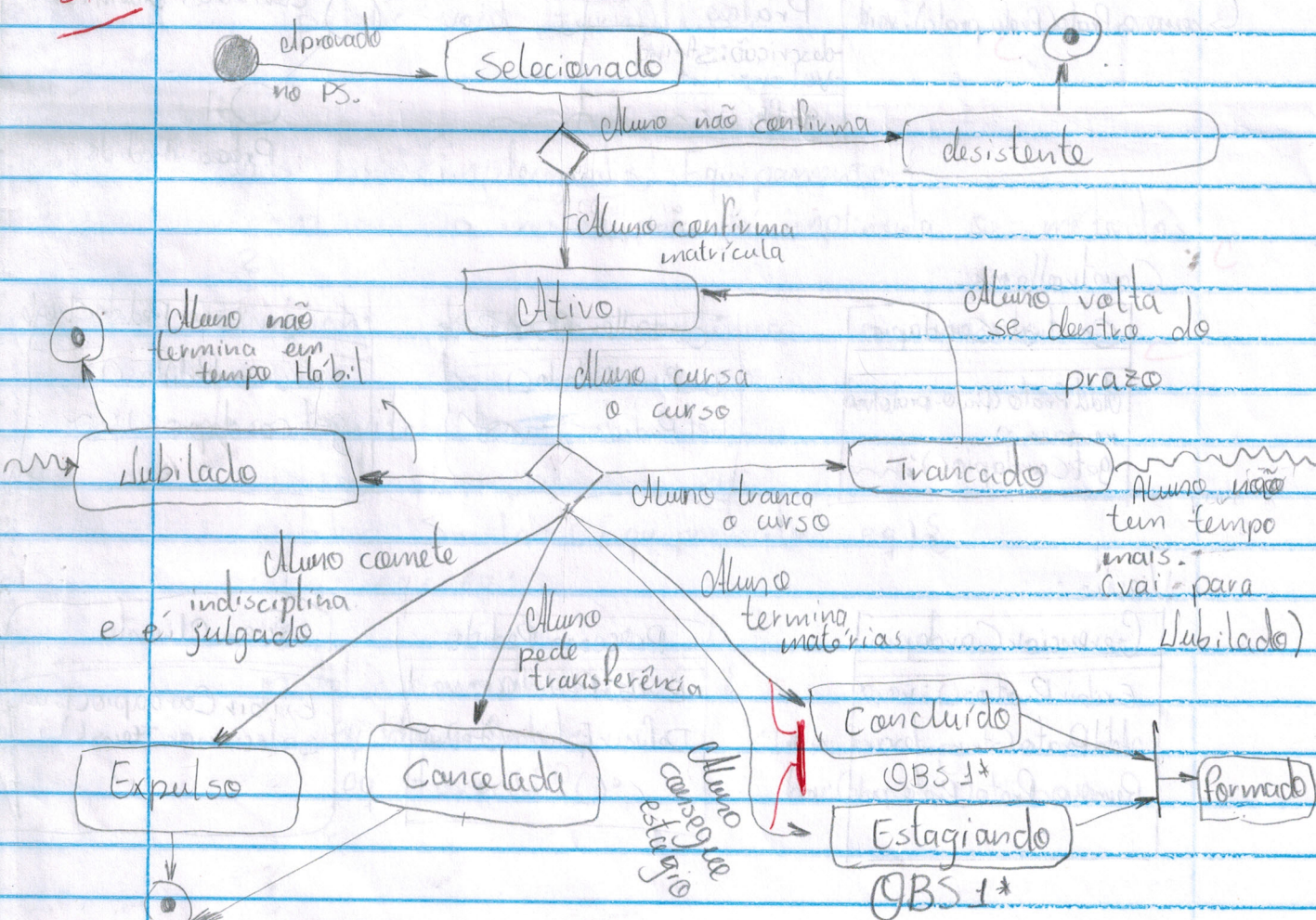
Public class Acrescimos extends Acrescimo {
  protected List <Acrescimos>
  // int getValorT() { // soma o List }
}
  
```

```

Public class AcrescimoLI {
  private descricao
  // setters, getters e construtor
}
  
```

ps: as caixinhas mostram o estado da matrícula

2.9

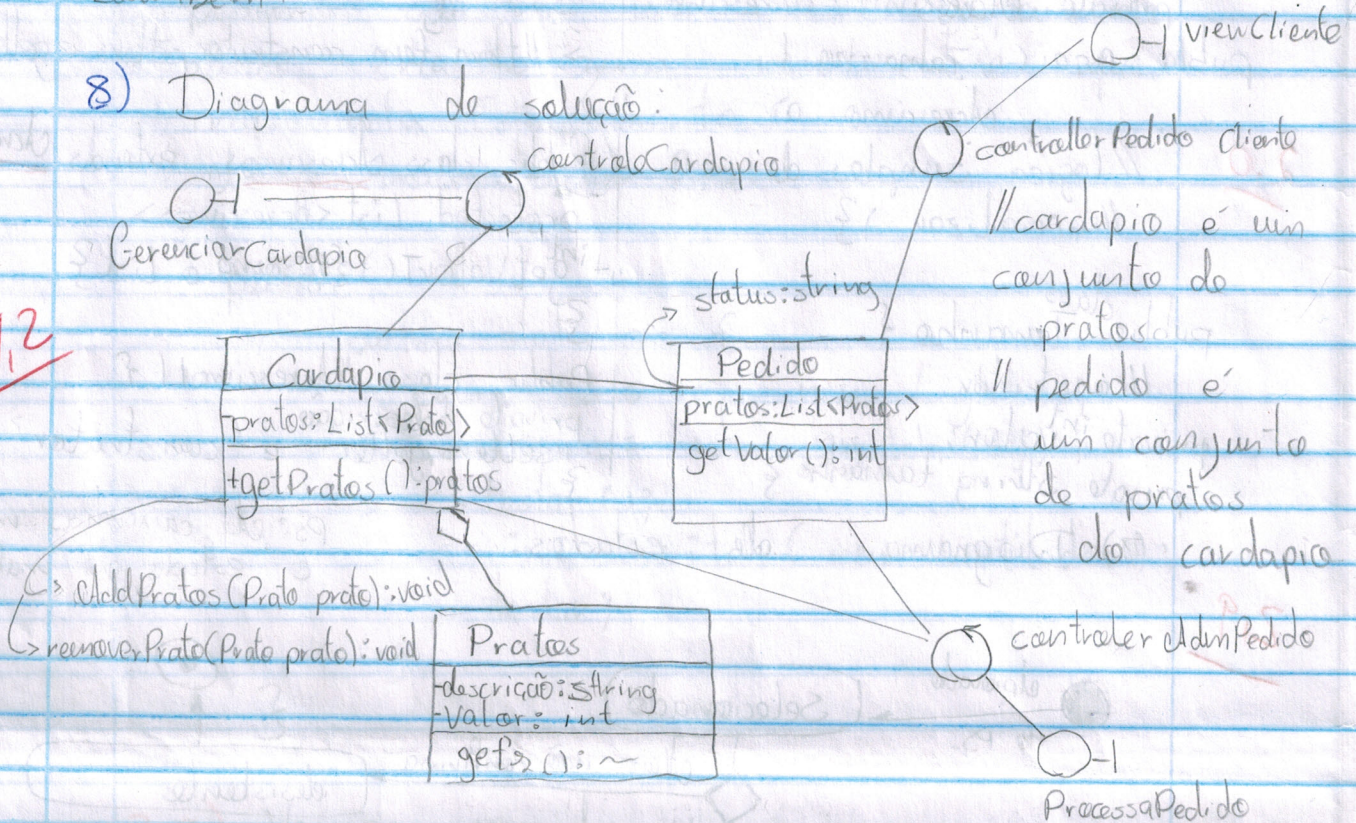


ou que concluiu as disciplinas

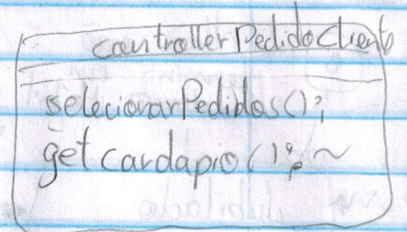
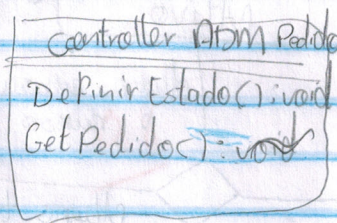
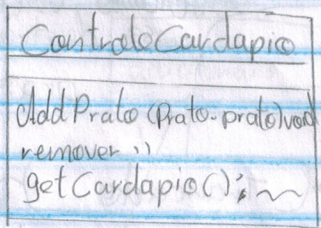
OBS 1: O aluno estagiando ainda pode ser expulso ou jubilado, então imagine as setinhas e para lá também.

8) Diagrama de solução:

4,2



Controllers:



Views:

